Shamir Secret Sharing

Distributed System Implementation

Team-10 members:

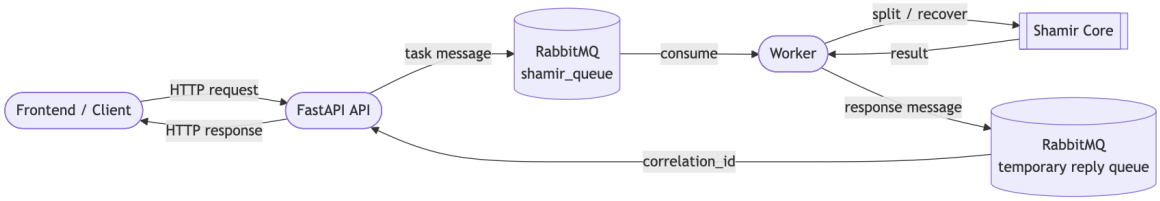
Karim Khabibrakhmanov : k.khabibrakhmanov@innopolis.university
Bogdan Zinchenko : b.zinchenko@innopolis.university
Artur Aipov : a.aipov@innopolis.university
Nikolay Taran : n.taran@innopolis.university
Ruslan Ustinov : r.ustinov@innopolis.university

Introduction

We implemented Shamir Secret Sharing for splitting a secret into multiple parts and reconstructing it from a threshold number of shares.

The goal was not only to implement the algorithm, but also to build a small distributed system using concepts from DNP.

Architecture



The API does not perform computations directly. Instead, it sends tasks to RabbitMQ, where a worker processes them. This simulates real distributed systems with separation of concerns.

Why Distributed

We intentionally avoided a simple synchronous design. Instead, we used RabbitMQ to practice:

- async processing

- queue-based communication
- service separation

How Shamir Works

Idea

We represent the secret as a value s and embed it into a polynomial:

$$f(x) = s + a_1 x + a_2 x^2 + \ldots + a_{k-1} x^{k-1}$$

$$share_i = (x_i,\ f(x_i) \text{ mod } p)$$

$$s = f(0)$$

The secret is never directly shared. Each share is just a point on the same polynomial.

Encoding (Splitting Secret)

We generate a random polynomial where:

- constant term = secret
- other coefficients = random
- all operations are done in finite field Z_p

```
prime, coefficients = generate(secret_int, threshold)

for x in range(1, total_shares + 1):
    y = calculate_polynomial(coefficients, x, prime)
```

Each computed point (x, y) becomes a share.

Decoding (Recovering Secret)

To recover the secret, we reconstruct the polynomial at $x = 0$ using Lagrange interpolation.

$$f(0) = \sum_{i=1}^{k} y_i \cdot \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \text{ mod } p$$

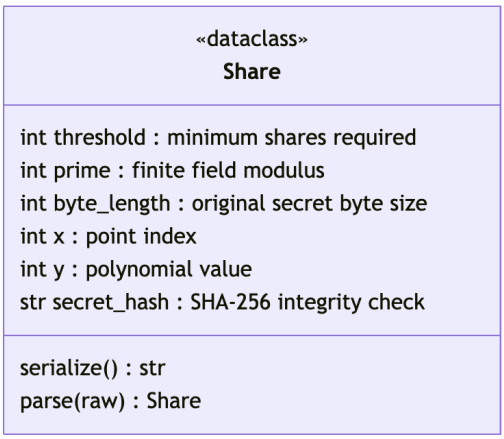
```python
secret = 0
for i, (x_i, y_i) in enumerate(points):
    numerator = 1
    denominator = 1

    for j, (x_j, _) in enumerate(points):
        if i != j:
            numerator = (numerator * (-x_j)) % prime
            denominator = (denominator * (x_i - x_j)) % prime

    basis = numerator * pow(denominator, -1, prime)
    secret = (secret + y_i * basis) % prime
```

We compute the value of the polynomial at zero without ever reconstructing the full polynomial explicitly.

Why It Works

- Any k points uniquely define the polynomial
- Fewer than k points give no information about s
- All computations are performed modulo a large prime p

Share Structure

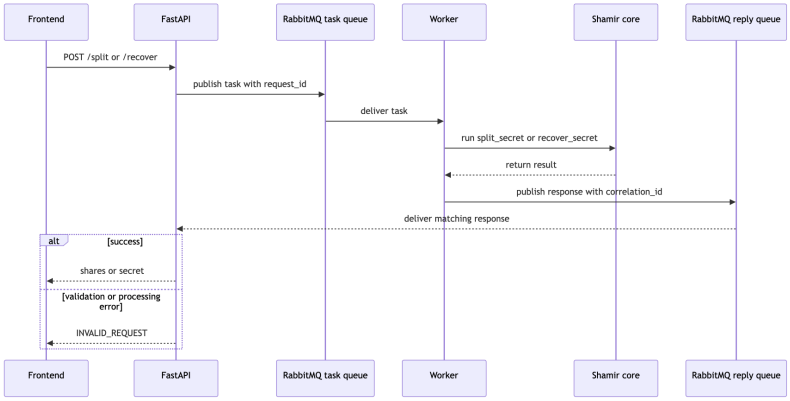


Each share contains all required metadata to reconstruct the secret:

- threshold
- prime (finite field)
- byte length
- x, y (point on polynomial)
- hash (SHA-256 for integrity)

The hash ensures that reconstructed data is valid and not corrupted.

Processing Flow



- Client sends request to API
- API publishes task to queue
- Worker consumes and processes it
- Result is sent back via reply queue
- API returns response to client

Validation & Error Handling

All user-facing errors are returned in a unified format.

```
{
  "code": "INVALID_REQUEST",
  "message": "error text",
  "request_id": "..."
}
```

Errors are grouped into three categories.

Input
- secret empty
- threshold < 2
- total_shares < 2
- threshold > total_shares
- invalid body

Reconstruction
- insufficient shares
- malformed share
- duplicate x
- inconsistent shares
- invalid shares

Infrastructure
- worker timeout

Hash validation ensures that even mathematically valid but corrupted results are rejected.

Team Work

- **Karim Khabibrakhmanov** — architecture, RabbitMQ, report
  Designed system architecture, implemented message broker flow and overall system design.
- **Bogdan Zinchenko** — core logic

Implemented polynomial generation and Shamir core logic.

- **Artur Aipov** — API
  Developed FastAPI backend and request handling.
- **Nikolay Taran** — core logic
  Worked on encoding/decoding and validation logic.
- **Ruslan Ustinov** — UI
  Implemented frontend and integration with backend.

Conclusion

We combined cryptography with distributed systems.

The project helped us understand how real systems separate computation and communication.